

Corso di Laurea in Ingegneria e Scienze Informatiche

Analisi dell'impiego di SOG e COG in algoritmi di clustering per reti marittime

Tesi di laurea in:
PROGRAMMAZIONE AD OGGETTI

Relatore

Prof. Danilo Pianini

Candidato

Lorenzo Antonioli

Correlatore

Dott. Martina Baiardi

Sommario

Max 2000 characters, strict.

Optional. Max a few lines.

Indice

Sommario	iii
1 Introduzione	1
2 Background	3
2.1 Comunicazioni marittime e AIS	3
2.1.1 Funzionamento del sistema AIS	4
2.1.2 Struttura e codifica del messaggio AIS	4
2.1.3 Aspetti normativi e applicazioni	6
2.1.4 Limiti e problemi aperti	6
2.2 Aggregate Programming, SCR e Kollektive	7
2.2.1 Programmazione Aggregata (AP)	7
2.2.2 Field Calculus (FC) e Building Blocks	8
2.2.3 Self-organising Coordination Regions (SCR)	10
2.2.4 Kollektive	11
2.3 Simulazione in ambito marittimo	14
2.3.1 Modeling & Simulation (M&S)	14
2.3.2 Alchemist	14
2.4 Algoritmo precedente	18
3 Analisi	21
3.1 Limiti dell'algoritmo esistente	21
3.2 Obiettivi	22
3.3 Requisiti	23
3.3.1 Requisiti funzionali	23
3.3.2 Requisiti non funzionali	23
4 Design e implementazione	25
4.1 Strategia di integrazione nel criterio di clustering	25
4.2 Estrazione e integrazione delle feature AIS aggiuntive	26
4.2.1 Decodifica messaggi AIS	27

4.2.2	Generazione tracce GPX	29
4.2.3	Caricamento di SOG e COG nella simulazione	29
4.3	Modifiche all'algoritmo	32
4.3.1	Costanti definite e lettura dei valori	32
4.3.2	Funzioni di calcolo della penalità	33
4.3.3	Integrazione nella metrica di selezione	34
5	Valutazioni	37
5.1	Risultati dell'esperimento precedente	37
5.2	Risultati del nuovo algoritmo	37
5.3	Confronto e discussione	37
6	Conclusioni e sviluppi futuri	39
6.1	Limitazioni dello studio	39
		45
	Bibliografia	45

Capitolo 1

Introduzione

Il settore marittimo rappresenta l'infrastruttura portante del commercio globale, ma si trova oggi ad affrontare una fase di profonda trasformazione tecnologica orientata all'efficienza, alla sicurezza e alla sostenibilità. In questo contesto, l'emergere delle Maritime Autonomous Surface Ships (MASS) promette di rivoluzionare la navigazione riducendo l'errore umano e ottimizzando i processi operativi [BPAFT25, AFBPT25]. Tuttavia, l'inadeguatezza delle reti di comunicazione marittima convenzionali frena il passaggio verso navi interamente o parzialmente autonome. Mentre i moderni dispositivi a bordo (come telecamere 4K, Light Detection and Ranging (LiDAR) e radar) generano massicci flussi di dati necessari per mantenere un'adeguata situational awareness, le comunicazioni tradizionali offrono una larghezza di banda limitata, insufficiente a supportare lo scambio di streaming video o di point clouds in tempo reale [BPAFT25, BPW14]. Per superare questo collo di bottiglia senza dipendere da costose connessioni satellitari, emerge la necessità di adottare paradigmi come Aggregate Programming, puntando a creare reti multi-hop e auto-organizzanti direttamente tra le imbarcazioni e le infrastrutture costiere [BPV15, CPVN19].

La motivazione alla base di questo lavoro di tesi nasce dall'analisi dei limiti delle attuali architetture di rete in ambito marittimo. L'articolo *Robust Communication through Collective Adaptive Relay Schemes for Maritime Vessels* ha valutato approcci in cui le navi si auto-organizzano in cluster, dove un leader comprime i flussi raccolti e inoltra l'output sintetizzato alla stazione a terra o al relay

successivo, sfruttando torri cellulari 5G [BPAFT25]. Sebbene questa architettura (identificata come la “Fase 1” dell’agenda *CoMPass*) migliori il data rate in scenari simulati, presenta una debolezza: la formazione dei cluster è basata solo su posizioni statiche [BPAFT25, AFBPT25]. L’uso di queste metriche non è infatti sufficiente, in quanto le imbarcazioni con traiettorie e velocità differenti dovrebbero essere scoraggiate dal formare collegamenti di comunicazione che diventerebbero rapidamente instabili [AFBPT25].

L’obiettivo principale di questo lavoro di tesi è implementare la “Fase 2” della roadmap CoMPass, ossia la riconfigurazione della rete consapevole della navigazione [AFBPT25]. Partendo dal framework di comunicazione stabilito, questa fase incorpora il posizionamento dinamico delle navi per ottimizzare la topologia della rete. Nello specifico, questa modifica utilizza i dati di navigazione estraendo dal payload dei messaggi Automatic Identification System (AIS) i valori di Speed Over Ground (SOG) e Course Over Ground (COG) per perfezionare gli algoritmi di clustering e le strategie di comunicazione. Valutando questi parametri, le navi che si muovono in direzioni e velocità simili ottengono la priorità per l’appartenenza al cluster mentre le imbarcazioni con traiettorie e velocità divergenti vengono scoraggiate dal formare collegamenti di comunicazione che diventerebbero rapidamente obsoleti.

Il documento è strutturato in capitoli secondo la seguente organizzazione. Il capitolo 2 espone i fondamenti teorici e le tecnologie di riferimento, descrivendo i sistemi di comunicazione marittima convenzionali (con particolare focus sulla struttura e i messaggi AIS), Aggregate Programming, Field Calculus e gli strumenti di simulazione utilizzati. Viene inoltre descritto in dettaglio l’esperimento precedente. Il capitolo 3 è dedicato all’analisi del problema. Vengono descritti: i limiti dell’algoritmo esistente, gli obiettivi della tesi e i requisiti funzionali e non funzionali richiesti per la realizzazione del nuovo sistema. Il capitolo 4 espone il design e l’implementazione, illustrando la metodologia di utilizzo e integrazione dei nuovi dati all’interno del programma e le modifiche all’algoritmo. Il capitolo 5 presenta la comparazione dei risultati ottenuti con quelli precedenti, utilizzando le stesse metriche di valutazione. Infine si discute delle conclusioni e dei possibili sviluppi futuri del lavoro svolto.

Capitolo 2

Background

In questo capitolo viene presentato il quadro teorico e tecnologico della tesi. La sezione 2.1 analizza i sistemi di comunicazione marittima convenzionali (con focus sull’AIS), evidenziando la differenza tra la loro banda limitata e l’elevata quantità di dati generata dalle navi a guida autonoma (MASS) [BPAFT25, AFBPT25].

Per superare questo limite, la sezione 2.2 introduce l’Aggregate Programming e il pattern Self-organising Coordination Regions (SCR) (tramite il framework Kollektive), soluzioni innovative per la creazione di reti auto-organizzanti [BPV15, CPVN19, BPAFT25].

Infine, la sezione 2.3 illustra il ruolo del Modeling & Simulation tramite il simulatore Alchemist. Poiché i test sul campo comportano limiti fisici ed elevati costi operativi, la simulazione offre l’ambiente virtuale ideale per sviluppare e valutare le prestazioni di tali reti cooperative [BPAFT25, PMV13].

2.1 Comunicazioni marittime e AIS

Le comunicazioni marittime si basano su una combinazione di sistemi radio Very High Frequency (VHF) e satellitari (Satellite Communications (SatCom)) per garantire la sicurezza della navigazione e l’efficienza delle operazioni [BPAFT25]. Mentre i sistemi satellitari assicurano una copertura globale, (comportando latenze e costi maggiori), i sistemi VHF supportano lo scambio di dati a corto raggio (in linea di vista o Line of Sight, tipicamente a 40 km a seconda del meteo) per i

collegamenti sia tra imbarcazioni (Ship to Vessel (S2V)) sia verso le stazioni costiere (Ship to Shore (S2S)) [BPAFT25]. Una delle tecnologie fondamentali basate sui sistemi radio è l'AIS. L'AIS è un sistema di identificazione automatica che permette alle navi di trasmettere e ricevere in modo continuo informazioni sulla propria posizione, rotta, velocità e stato di navigazione, utilizzando la banda radio VHF dedicata [Int26].

2.1.1 Funzionamento del sistema AIS

L'AIS è nato come strumento per aumentare la sicurezza in mare, affiancando i radar per ridurre il rischio di collisioni e migliorare la “consapevolezza situazionale” dei comandanti [Int26, Uni15]. Ogni transponder AIS invia periodicamente messaggi brevi (tipicamente ogni 2–6 secondi) che includono sia dati statici (identificativo International Maritime Organization (IMO), Maritime Mobile Service Identity (MMSI), tipo di nave, dimensioni) sia dati dinamici (posizione Global Positioning System (GPS), rotta, velocità, stato di navigazione), oltre a dati di viaggio come la destinazione prevista [Int26]. Questi messaggi possono essere letti da altri transponder a bordo (modalità S2V) e da stazioni costiere (S2S), permettendo il monitoraggio del traffico da parte delle autorità (Vessel Traffic Service (VTS), Guardia Costiera) [Int26, Uni15] e anche da servizi commerciali (es. portali come MarineTraffic¹ o VesselFinder²). La portata tipica è di alcune decine di miglia nautiche, limitata dalla propagazione VHF e dalle condizioni meteorologiche [BPAFT25], ma può essere estesa tramite reti satellitari Satellite AIS (AIS-sat) [Int26].

2.1.2 Struttura e codifica del messaggio AIS

I messaggi AIS non vengono trasmessi in chiaro come testo, ma sono codificati in formato binario compresso (utilizzando una codifica a 6 bit) per minimizzare l'occupazione della banda radio VHF [Int26]. I diversi tipi di messaggi (da 1 a 27) rispondono a specifiche esigenze; tra questi, i messaggi di Tipo 1, 2 e 3 sono i più frequenti e vengono impiegati dalle navi di Classe A per comunicare periodicamente

¹<https://www.marinetraffic.com>

²<https://www.vesselfinder.com>

i dati di posizione dinamica [Int26]. La tabella 2.1 descrive dettagliatamente la ripartizione dei bit all'interno di un frame AIS di Posizione standard.

Bit	Nome Campo	Descrizione
6	Message ID	Identifica il tipo di messaggio (valori: 1, 2 o 3).
2	Repeat Indicator	Numero di ripetizioni del messaggio.
30	Source ID (MMSI)	Codice univoco di 9 cifre della nave.
4	Navigational Status	Stato della nave (es. 0 = in navigazione, 1 = all'ancora, 5 = ormeggiata).
8	ROT	Velocità di accostata (virata) a destra o sinistra.
10	SOG	Velocità rispetto al fondo (risoluzione 0.1 nodi).
1	Position Accuracy	Accuratezza del GPS (1 = alta, ≤ 10 m; 0 = bassa, > 10 m).
28	Longitude	Longitudine in 1/10000 di minuto d'arco.
27	Latitude	Latitudine in 1/10000 di minuto d'arco.
12	COG	Rotta rispetto al fondo (in decimi di grado, 0–3599).
9	True Heading	Prua vera della nave (0–359 gradi).
6	Time Stamp	Secondo della coordinata temporale UTC di generazione (0–59).
2	Special Manoeuvre	Indicatore di manovre speciali.
2	Spare	Bit di riserva inutilizzati (impostati a 0).
1	Transmit Power	Potenza di trasmissione.
1	RAIM flag	Indicatore di integrità del ricevitore GPS.
19	Communication State	Dati di sincronizzazione e gestione degli slot temporali.
168	Totale	Struttura che occupa 1 slot di trasmissione.

Tabella 2.1: Struttura del payload di un messaggio AIS di Posizione (Tipo 1, 2, 3).

2.1.3 Aspetti normativi e applicazioni

L'uso del Sistema di Identificazione Automatica è regolato dall'IMO ed è obbligatorio ai sensi della convenzione Safety Of Life At Sea (SOLAS) per le navi commerciali in viaggio internazionale con stazza lorda pari o superiore a 300 Gross Tonnage (GT), e per tutte le navi passeggeri indipendentemente dalle loro dimensioni [BPW14, Uni15]. Oltre all'identificazione e al monitoraggio dei corridoi marittimi, l'AIS è oggi impiegato in diversi ambiti, quali:

- la prevenzione delle collisioni in tempo reale (Closest Point of Approach (CPA)), che permette ai sistemi di bordo di innescare allarmi visivi e acustici qualora si rilevi una possibile collisione [BPW14];
- le operazioni di ricerca e soccorso (Search and Rescue (SAR)) e la gestione di uomini in mare (Man OverBoard (MOB)): i trasmettitori di soccorso (es. AIS-Search and Rescue Transponder (AIS-SART)) si attivano a contatto con l'acqua trasmettendo la posizione GPS a tutte le unità limitrofe, utilizzando messaggi specifici e un prefisso MMSI dedicato [BPW14, Int26];
- la segnalazione di ostacoli tramite ausili alla navigazione (Aids to Navigation (AtoN)): boe, fari o stazioni possono trasmettere le proprie coordinate o simulare ostacoli virtuali (Virtual AtoN) per deviare il traffico da aree pericolose o fondali bassi [BPW14, Int26];
- il monitoraggio ambientale, ad esempio per rintracciare le navi responsabili di versamenti illegali di petrolio in mare [BPW14].

2.1.4 Limiti e problemi aperti

Nonostante l'ampia diffusione, sono presenti alcuni limiti e vulnerabilità dell'AIS:

- natura non autenticata e vulnerabilità informatica: i dati generati a bordo possono essere: manipolati (*spoofed*) per creare navi inesistenti o falsi allarmi SAR/CPA, alterati sovrascrivendo i segnali (*hijacking*), oppure resi indisponibili tramite attacchi radio *Denial of Service*, come il blocco del salto di frequenza (*frequency hopping*) o l'esaurimento degli slot disponibili (*slot starvation*) [BPW14];

- limiti fisici della trasmissione VHF: il canale offre una larghezza di banda molto stretta (circa 10 kbps) a fronte di una copertura limitata Line of Sight (tipicamente 40 km, fortemente dipendente dalle condizioni meteorologiche) [BPAFT25];
- incompatibilità con i requisiti delle navi a guida autonoma (MASS): questi mezzi richiedono sensori avanzati come telecamere ad alta risoluzione, LiDAR e radar, generando flussi di dati continui nell'ordine dei Mbps che le attuali reti marittime non sono in grado di sostenere [BPAFT25].

In particolare, questo lavoro di tesi affronterà l'ultimo problema elencato sopra, utilizzando la programmazione aggregata (sezione 2.2).

2.2 Aggregate Programming, SCR e Kollektive

2.2.1 Programmazione Aggregata (AP)

La programmazione di sistemi distribuiti moderni, come l'Internet of Things (IoT) o le reti navali, risulta complessa a causa dell'eterogeneità dei dispositivi, dei guasti frequenti e della necessità di interazioni basate sulla prossimità fisica [BPV15]. Per far fronte a questi problemi, Aggregate Programming (AP) propone un cambio di paradigma: l'unità base della computazione non è più il singolo dispositivo, ma un intero insieme (o aggregato) di nodi cooperanti, astruendo dettagli come il numero e l'esatta posizione dei singoli apparati [BPV15, BV16]. AP è un approccio di macro-programmazione funzionale basato sul concetto di campo computazionale (computational field), ovvero una struttura dati distribuita che mappa ogni dispositivo in un dato spazio e tempo a un valore [VAB⁺18, BPAFT25]. I dispositivi eseguono tutti un medesimo programma in modo asincrono (in round continui) seguendo un modello a tre fasi [BPAFT25]:

- *sense*: il nodo raccoglie informazioni dall'ambiente locale e dai messaggi ricevuti dai vicini durante la fase di riposo;
- *compute*: il programma aggregato viene valutato usando i dati raccolti, producendo un nuovo stato locale e i messaggi da condividere;

- *share*: il dispositivo invia i messaggi ai propri vicini.

2.2.2 Field Calculus (FC) e Building Blocks

Il fondamento matematico di Aggregate Programming è Field Calculus (FC), un calcolo universale minimale che modella le interazioni dei dispositivi nello spazio e nel tempo [BV16, VAB⁺18]. FC opera a un basso livello di astrazione, basandosi su un numero ridotto di costrutti [VAB⁺18]:

- **nbr** (*Neighborhood values*): permette l'interazione del dispositivo con il vicinato, raccogliendo e mappando i valori più recenti di una determinata espressione provenienti dai nodi vicini e dal nodo stesso [VAB⁺18, BV14];
- **rep** (*Time evolution*): modella la dinamica temporale, inizializzando una variabile di stato locale e aggiornandola ciclicamente a ogni round di computazione [VAB⁺18, BV14];
- **if** (*Domain restriction*): partiziona la rete in sottoregioni in cui vengono eseguiti rami di codice differenti [VAB⁺18].

Per aumentare il livello di astrazione ed evitare la gestione manuale dei costrutti come **rep** e **nbr**, come illustrato in fig. 2.1, sono stati identificati una serie di building blocks formalmente provati per essere auto-stabilizzanti. Questi blocchi incapsulano i costrutti di FC e ciò garantisce che il sistema che li utilizza sia in grado di recuperare da guasti e adattarsi ai cambiamenti topologici, convergendo sempre verso lo stato corretto [BV14, VAB⁺18]. I principali operatori di base sono [BV14, BPV15]:

- **G**: diffonde informazioni da una regione sorgente seguendo un gradiente (es. calcolo delle distanze minime o broadcast);
- **C**: accumula informazioni muovendosi lungo il gradiente di un potenziale (tipicamente verso un nodo pozzo o sink);
- **T**: regola l'evoluzione temporale, agendo come una memoria a tempo o un count-down;

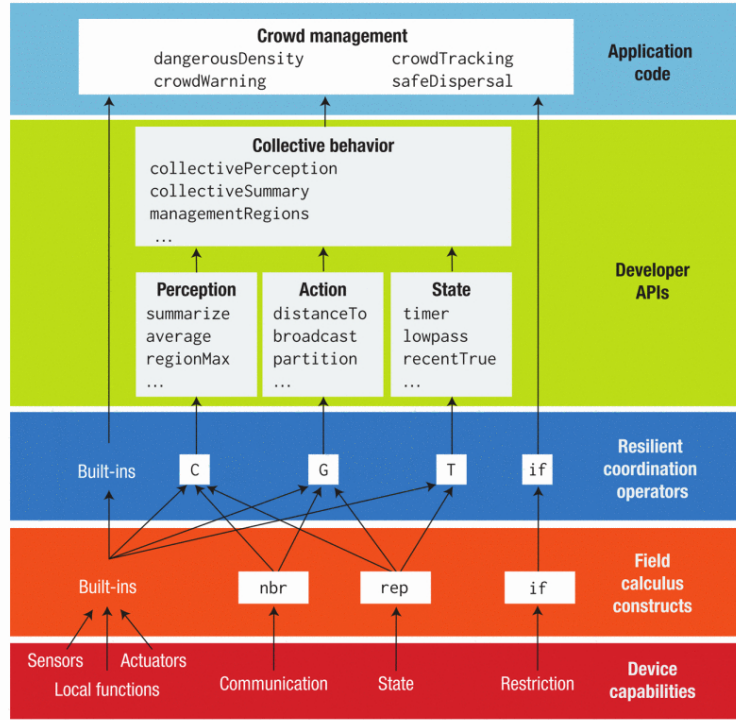


Figura 2.1: Livelli di astrazione della programmazione aggregata. Le capacità software e hardware di alcuni dispositivi sono utilizzate per implementare costrutti di FC a livello aggregato. Questi costrutti sono utilizzati per implementare operazioni di coordinamento a blocchi resilienti, che vengono poi racchiuse e combinate insieme per produrre API. © 2015 IEEE. Ristampata, con permesso, da [BPV15].

- **S**: elegge dei leader locali in modo sparso, frammentando lo spazio in partizioni (es. diagrammi di Voronoi).

2.2.3 Self-organising Coordination Regions (SCR)

Sfruttando i building blocks, è stato ideato un design pattern chiamato SCR, pensato appositamente per gestire reti molto grandi e complesse, come quelle dell'Edge Computing [CPVN19]. Il suo obiettivo è trovare un compromesso efficace tra un controllo totalmente centralizzato e uno completamente decentralizzato [CPVN19]. Il sistema si auto-organizza suddividendosi spontaneamente in regioni. All'interno di ciascuna regione, le attività vengono coordinate in modo autonomo grazie a un continuo scambio di informazioni (un ciclo di feedback, come mostrato in fig. 2.2) tra un nodo responsabile (leader) e i dispositivi circostanti [CPVN19]. Il funzionamento di questo pattern si basa su quattro processi continui [CPVN19]:

1. *Elezione dei leader*: vengono eletti i leader da un insieme di candidati (tramite l'operatore S).
2. *Formazione delle regioni*: ogni nodo viene assegnato a un singolo leader, costruendo i percorsi di comunicazione ottimali.
3. *Raccolta delle informazioni*: flussi di dati si muovono dai nodi della regione verso il leader, utilizzando algoritmi come C.
4. *Propagazione delle informazioni*: i leader valutano i dati raccolti e diffondono i risultati o le direttive verso tutti i membri della propria regione (utilizzando G).

Il pattern SCR è resiliente: se un leader subisce un guasto o si verificano variazioni nel carico, il sistema riconfigura automaticamente i leader e ridimensiona le regioni in pochi secondi [CPVN19].

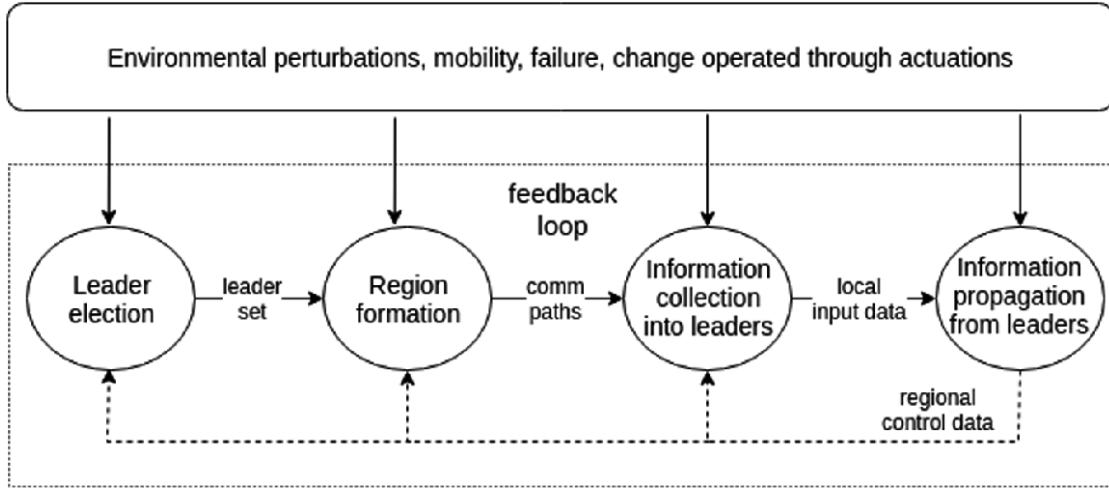


Figura 2.2: Prospettiva dinamica di SCR [CPVN19].

2.2.4 Collektive

Dal punto di vista dell'implementazione, la programmazione aggregata viene concretizzata tramite Domain Specific Languages (DSL), tra cui ScaFi³ (basato su Scala) e Protelis⁴ (basato su Java). In questa tesi viene utilizzato **Collektive**⁵, un DSL nativo basato su Kotlin. È un framework open-source progettato per semplificare lo sviluppo di sistemi distribuiti attraverso l'utilizzo dei principi di AP [Col26]. Architetturealmente, Collektive è suddiviso in tre macro-componenti:

1. un plugin per il compilatore, che si occupa di gestire l'allineamento;
2. il DSL, che definisce i costrutti linguistici fondamentali;
3. e la libreria standard, che fornisce le funzioni essenziali per AP.

Il DSL Collektive espone funzioni per la gestione dello stato e delle interazioni di rete, ad esempio [BPAFT25]:

- **neighboring**: consente di osservare e mappare in un campo i valori di una specifica variabile nei nodi vicini. Corrisponde a `nbr` in FC;

³<https://scafi.github.io/>

⁴<https://protelis.github.io/wiki-beta/>

⁵<https://collektive.github.io>

- **exchange**: è uno degli operatori più importanti del DSL. Modella la comunicazione *anisotropica*, consentendo a un nodo di inviare messaggi differenti a vicini diversi.
- **share**: modella la comunicazione *isotropica* (invia lo stesso stato a tutti), combinando lo stato precedente con le informazioni scambiate nel vicinato per evolvere i dati nello spazio e nel tempo;
- **evolve**: equivale a **rep** nel FC e consente di modellare l'evoluzione dello stato del dispositivo nel tempo;
- **alignedOn**: introduce la segmentazione del dominio (domain segmentation), isolando le comunicazioni ed eseguendo i blocchi di codice esclusivamente tra i dispositivi che soddisfano e condividono un medesimo identificatore (pivot) [BPAFT25].

I building blocks vengono implementati componendo queste primitive. Operazioni come **gradientCast** o il calcolo delle distanze tramite Bellman-Ford si ottengono combinando **exchange** o **share** con una metrica, ad esempio calcolando la formula di Haversine sui dati GPS recuperati tramite **neighboring** [BPAFT25].

Per quanto concerne l'implementazione del pattern SCR, Collektive vi provvede utilizzando algoritmi di elezione e segmentazione. La formazione dinamica delle regioni è delegata alla funzione **boundedElection**, che partiziona la rete senza alcuna coordinazione centrale [BPAFT25]. L'algoritmo riceve in input il diametro massimo del cluster e una metrica di “forza” del nodo (la priorità del candidato). Questa “forza” coincide con l'eleggibilità del dispositivo a diventare leader (che può basarsi su risorse energetiche, di calcolo, o sul data rate come nel caso dell'esperimento iniziale di questa tesi [BPAFT25]). È possibile personalizzare l'elezione del leader utilizzando **Candidacy** (rappresenta la candidatura in un'elezione di un candidato) all'interno del parametro **selectBest** in **boundedElection**. Valutando le “forze”, **boundedElection** risolve le contese ed elegge i leader, adattandosi tempestivamente ai guasti o ai cambiamenti topologici [BPAFT25]. Una volta eletti i leader, all'interno della regione viene utilizzato l'operatore **alignedOn(leader)**: in questo modo i dispositivi di una stessa regione restringono il proprio dominio alla sola regione di appartenenza, instaurando canali diretti (come dei **convergeCast**)

Listing 2.1: Esempio di codice Collektive per una semplice simulazione di programmazione aggregata.

```
1 fun Aggregate<Int>.temperatureEntrypoint(env: EnvironmentVariables,
2   collektiveDevice: CollektiveDevice<*>): Boolean {
3   val temp: Double = env["temperature"]
4   val distances = with(collektiveDevice) { distances() }
5   val isLeader = (boundedElection(bound = 20.0, metric = distances) == localId)
6   .also { env["leader"] = it }
7   val closestLeader = gradientCast(source = isLeader, local = localId, metric =
8     distances)
9   return alignedOn(closestLeader) {
10    val regDistances = with(collektiveDevice) { distances() }
11    val count = countDevices(sink = isLeader)
12    val sum = convergeSum(sink = isLeader, local = temp)
13    val avg = if (isLeader && count > 0) sum / count else 0.0
14    val alarmDecision = isLeader && avg > 30.0
15    gradientCast(source = isLeader, local = alarmDecision, metric =
16      regDistances)
17    .also { env["alarm"] = it }
18  }
19 }
```

per instradare le informazioni senza interferire con le regioni adiacenti [BPAFT25]. `temperatureEntrypoint` (listing 2.1) rappresenta un esempio di un semplice algoritmo in cui vengono utilizzate le nozioni di SCR e AP: i nodi possiedono un attributo “temperatura”. Essi si organizzano in regioni con il proprio leader e quest’ultimo si occupa di calcolare la media delle temperature dei dispositivi della propria area. Se la media supera i 30°C allora il leader diffonde un allarme.

2.3 Simulazione in ambito marittimo

2.3.1 Modeling & Simulation (M&S)

I porti marini e gli scenari di navigazione costiera sono sistemi altamente complessi, caratterizzati da un'elevata densità di traffico, condizioni meteo variabili e diversi attori (navi mercantili, piccoli natanti, rimorchiatori, e torri di controllo). In tale contesto, testare nuove configurazioni di rete, valutare l'efficienza delle operazioni o formare il personale sul campo risulta spesso proibitivo per questioni di sicurezza, limiti fisici ed elevati costi operativi [CPL15].

Per risolvere queste problematiche, il paradigma Modeling & Simulation (M&S) rappresenta lo strumento d'eccellenza. M&S consente di riprodurre il comportamento del sistema reale attraverso un modello computazionale, offrendo un ambiente virtuale in cui analizzare scenari “what-if”, condurre analisi sugli incidenti e massimizzare l'efficacia del training [CPL15]. In particolare, la simulazione distribuita permette l'addestramento congiunto di piloti di navi e operatori del traffico (VTS) all'interno di un unico ambiente virtuale condiviso, riducendo i rischi e i costi associati agli errori in fase di apprendimento [CPL15]. Oltre alla formazione del personale, la simulazione si rivela oggi un mezzo fondamentale per lo sviluppo e la prototipazione di reti di telecomunicazione e schemi di coordinamento adattivo destinati alle future navi a guida autonoma (MASS) [BPAFT25, CPL15].

2.3.2 Alchemist

Per l'esecuzione e la valutazione dell'esperimento di questa tesi è stato adottato **Alchemist**⁶, una piattaforma di simulazione di sistemi distribuiti, particolarmente adatta alla programmazione aggregata. Consente di modellare e simulare le interazioni tra agenti in ambienti complessi, supportando diverse astrazioni e paradigmi di programmazione [Col26].

⁶<https://alchemistsimulator.github.io/>

Meta-modello e Astrazioni

Per garantire la flessibilità necessaria a simulare scenari complessi, la logica di Alchemist, come mostrato in fig. 2.3, si basa su un meta-modello le cui entità fondamentali sono [Pia21]:

- *Environment*: lo spazio in cui vivono i nodi, con la responsabilità di mapparne la posizione. Può includere ostacoli fisici, mappe reali (es. file OpenStreetMap⁷ o tracce GPS) e ambienti indoor [Pia21].
- *Node*: un contenitore situato nell'ambiente che ospita reazioni e "molecole".
- *Linking rule*: una funzione dell'ambiente che associa ogni nodo a un vicinato.
- *Molecule* e *Concentration*: la molecola rappresenta un dato, mentre la concentrazione ne rappresenta il valore associato all'interno di uno specifico nodo.
- *Reaction*: un evento che modifica lo stato dell'ambiente. È composto da condizioni (che ne determinano l'eseguibilità), una distribuzione temporale, un'equazione di tasso (*rate*) e azioni concrete (es. movimento del nodo o alterazione di una concentrazione) [PMV13, Pia21].

⁷<https://www.openstreetmap.org/>

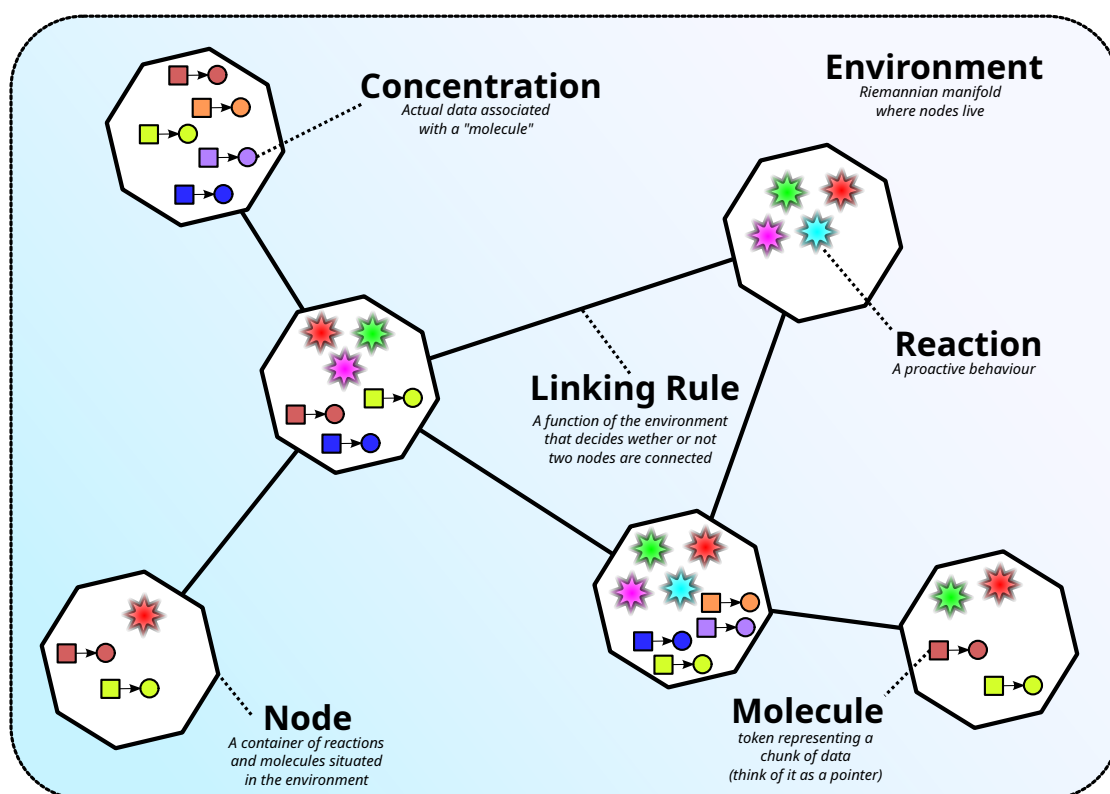


Figura 2.3: Rappresentazione grafica del meta-modello di Alchemist [Pia21].

Incarnazioni (*Incarnations*) per Aggregate Programming

Per supportare linguaggi e paradigmi differenti, Alchemist sfrutta il concetto di *Incarnation* (incarnazione), che permette di definire il tipo di dato esatto usato per le variabili (Concentrazione) e di fornire comportamenti specifici [Pia21]. Tra le incarnazioni supportate, assumono particolare rilevanza: `collektive`, `protelis` e `scafi`. Queste incarnazioni sono progettate specificamente per eseguire programmi basati su AP [Pia21]. Insieme, Alchemist e Collektive, consentono di implementare programmi aggregati in Kotlin che possono essere eseguiti e simulati in ambienti dinamici e configurabili [Col26].

Configurazione e Riproducibilità

Alchemist semplifica la configurazione degli ambienti di test affidandosi a specifiche scritte in *YAML*, un linguaggio di serializzazione dati in formato dichiarativo [Pia21]. Tramite un file *YAML* è possibile definire per esempio i modelli di rete (*network-model*) e la disposizione iniziale dei nodi (*deployments*). In listing 2.2 viene mostrata la struttura di una possibile configurazione di una simulazione, riprendendo l'esempio: listing 2.1. Un aspetto pratico fondamentale di Alchemist per le simulazioni in ambito marittimo è il supporto nativo a dati geospaziali. Il simulatore permette di importare ambienti fisici basati su OpenStreetMap e di impostare la dinamica dei nodi importando direttamente tracce GPS in formato GPS eXchange Format (GPX) (generate, ad esempio, dalla decodifica di messaggi AIS), consentendo di testare algoritmi sulle rotte effettivamente percorse dalle imbarcazioni [Pia21, BPAFT25]. Infine, Alchemist garantisce la ripetibilità degli esperimenti: il sistema separa e controlla in modo deterministico i seed (semi) dei generatori pseudo-casuali (come il Mersenne Twister) sia per la disposizione iniziale dei nodi, sia per l'esecuzione della simulazione [Pia21].

Listing 2.2: Parte della configurazione della simulazione per il programma: listing 2.1.

```
1 incarnation: collective
2
3 network-model:
4   type: ConnectWithinDistance
5   parameters: [3.0]
6
7 _pool: &program
8   - time-distribution: 1
9     type: Event
10    actions:
11      - type: RunCollectiveProgram
12        parameters: [it.unibo.collective.TemperatureKt.temperatureEntrypoint]
13
14 deployments:
15   - type: Grid
16     parameters: [ 0, 0, 40, 80, 2, 2, 0.5, 0.5 ]
17     programs:
18       - *program
19     contents:
20       - molecule: leader
21         concentration: false
22       - molecule: temperature
23         concentration: 17.0
```

2.4 Algoritmo precedente

L'esperimento su cui si basa questa tesi ha come obiettivo quello di migliorare la raccolta e la trasmissione di dati in uno scenario marittimo realistico. Il caso di studio considera il fiordo di Kiel, in Germania, dove le navi sono modellate come nodi in grado di comunicare tra loro e con stazioni a terra [BPAFT25]. Il problema affrontato nasce dal divario tra la quantità di dati generata dai moderni sistemi di navigazione autonoma (MASS) e la capacità delle infrastrutture di comunicazione marittima tradizionali. Navi autonome o semi-autonome possono infatti produrre flussi di dati ad alta intensità, derivanti ad esempio da telecamere e LiDAR; al contrario, tecnologie tradizionali come AIS e sistemi basati su VHF offrono una capacità trasmissiva limitata [BPAFT25]. Nella simulazione, ogni imbarcazione è dotata di diversi dispositivi di comunicazione: un sistema Automatic Packet Reporting System (APRS) su VHF, un dispositivo Long Range Wide Area Network (LoRaWAN) di classe C, un modulo 5G di tipo consumer e un dispositivo compatibile con Wi-Fi Direct [BPAFT25]. Inoltre, alcune imbarcazioni possono essere equipaggiate con una torre 5G montata a bordo, con probabilità indicata

dal parametro p_{5G} . Ogni nave tenta di trasmettere verso terra dei dati di dimensione fissata pari a $d = 3$ Mbit/s, valore scelto come approssimazione del bitrate necessario per un flusso video compresso a 720p [BPAFT25].

Per stimare la qualità dei collegamenti, l'esperimento associa a ogni coppia di nodi un data rate stimato in funzione della tecnologia disponibile e della distanza tra i dispositivi. Le tecnologie considerate sono quindi 5G, Wi-Fi, LoRaWAN e APRS. A partire da valori di riferimento ricavati dalla letteratura, il data rate in linea di vista viene stimato mediante interpolazione log-lineare rispetto alla distanza [BPAFT25].

Gli approcci confrontati nell'esperimento precedente sono quattro:

- *Baseline non cooperativa*: rappresenta lo stato dell'arte di riferimento. Ogni nave tenta di comunicare direttamente con una stazione a terra utilizzando la tecnologia più favorevole tra quelle disponibili. Se il collegamento diretto offre un data rate sufficiente a trasmettere il flusso d , la comunicazione viene considerata riuscita; in caso contrario, il flusso viene scartato [BPAFT25]. Questo approccio non prevede alcuna collaborazione tra imbarcazioni.
- *Distance-based Multi-Relay Communication (Dist-MR)*: ogni nave che non riesce a raggiungere direttamente la stazione a terra seleziona come relay un'imbarcazione vicina lungo il cammino geograficamente più breve verso la destinazione. Si ha quindi un insieme di alberi che originano nelle stazioni a terra. La metrica usata per la scelta del relay è la distanza geografica [BPAFT25].
- *Data Rate-based Multi-Relay Communication (DR-MR)*: modifica il criterio di scelta del relay utilizzato in Dist-MR. Invece di minimizzare la distanza geografica, l'obiettivo è selezionare il percorso che massimizza il data rate disponibile verso la stazione a terra [BPAFT25].
- *Collective Summarisation Clusters (CSC)*: organizza le imbarcazioni in cluster auto-organizzanti. Ogni cluster possiede un leader, responsabile della raccolta dei flussi dati prodotti dai membri della regione e della loro successiva sintesi prima dell'invio verso terra o verso un altro cluster. Questo approccio si basa sul pattern SCR: i leader vengono eletti dinamicamente e

la formazione dei cluster tiene conto della capacità di trasmettere dati verso il leader [BPAFT25].

Capitolo 3

Analisi

3.1 Limiti dell'algoritmo esistente

L'algoritmo analizzato nel capitolo precedente (sezione 2.4) organizza le navi in cluster valutandone principalmente la posizione e il canale di comunicazione con il leader. In particolare, vengono utilizzati la longitudine e la latitudine delle imbarcazioni e il data rate [BPAFT25]. Questo approccio non tiene conto delle dinamiche complesse che caratterizzano l'ambiente marittimo. Se in un dato istante due nodi sono fisicamente vicini, potrebbero non esserlo più poco tempo dopo a causa della loro elevata differenza di rotta o velocità. Raggrupparli in uno stesso cluster comporterebbe una rapida rottura del collegamento, il che innescerebbe continue rielezioni del leader o dei relay e frammentazioni della rete.

Dunque, il principale limite dell'algoritmo attuale risiede nel mancato sfruttamento dei dati a disposizione nel payload AIS. Come illustrato nella sezione 2.1.2, i messaggi del Sistema di Identificazione Automatica (in particolare quelli di tipo 1, 2 e 3) contengono diverse informazioni di navigazione, tra cui: SOG (velocità), COG (direzione) e ROT (velocità di virata) [Int26]. Al momento, l'algoritmo ignora completamente questi dati, limitandosi a estrarre solo la posizione.

Altre limitazioni dell'esperimento sono:

- *quantità modesta di dati*: i dati utilizzati per l'esperimento provengono da uno scenario reale nel fiordo di Kiel. Tuttavia, riguardano una fine-

stra circoscritta a sei ore nell'arco di una giornata, che potrebbe non essere rappresentativa del traffico nel fiordo [BPAFT25];

- *casi limite*: non sono stati valutati alcuni casi limite (es. guasti al nodo leader) per esaminare il comportamento degli approcci valutati in specifiche situazioni [BPAFT25];
- *sicurezza*: l'esperimento non tiene conto degli aspetti sulla sicurezza trattati nel primo punto della sezione 2.1.4 [BPAFT25];
- *sintesi dei dati*: non sono stati ricercati algoritmi di sintesi dati [BPAFT25];
- *condizioni meteo*: non sono state prese in considerazione le avversità e il mutare delle condizioni meteo marine [BPAFT25].

3.2 Obiettivi

L'obiettivo di questa tesi (in linea con le direttive della Fase 2 della roadmap *CoMPass* [AFBPT25]) è quello di superare i limiti dell'approccio che tiene conto solo della posizione, migliorando la selezione dei relay e la formazione dei cluster mediante l'inclusione di dati di navigazione aggiuntivi. Nello specifico, si intende estendere l'algoritmo CSC estraendo dal payload AIS i dati di SOG (velocità rispetto al fondo) e COG (rotta rispetto al fondo) per integrarli direttamente all'interno dell'algoritmo. Due navi che, in un determinato istante, risultano geograficamente vicine e presentano un buon data rate possono essere assegnate alla stessa regione. Tuttavia, se tale scelta non considera le differenze di direzione e velocità, le imbarcazioni potrebbero allontanarsi rapidamente rendendo necessarie frequenti riorganizzazioni della rete. In base a quanto le imbarcazioni viaggiano in direzioni simili e velocità affini, si punta a creare cluster più stabili nel tempo, riducendo la volatilità della topologia della rete e migliorando l'efficienza della comunicazione verso la costa.

3.3 Requisiti

Per garantire il raggiungimento degli obiettivi prefissati, il nuovo algoritmo deve soddisfare una serie di requisiti. Tali requisiti sono elencati di seguito suddivisi in funzionali (le operazioni che il sistema deve compiere) e non funzionali (come le deve compiere).

3.3.1 Requisiti funzionali

- Il sistema deve poter estrarre dai messaggi AIS la velocità e la rotta.
- In assenza delle informazioni cinematiche, deve essere possibile continuare a operare senza tali dati.
- L'algoritmo deve valutare le differenze di SOG e COG tra imbarcazioni e favorire, nella scelta dei relay o nella formazione dei cluster, nodi con una mobilità compatibile con la nave presa in esame.
- La simulazione deve essere in grado di esporre i dati ottenuti per poter effettuare valutazioni e confronti con la versione precedente dell'algoritmo.

3.3.2 Requisiti non funzionali

- Il calcolo delle nuove metriche con l'aggiunta di SOG e COG non deve introdurre un carico computazionale tale da far perdere i vantaggi del CSC.
- Le simulazioni devono poter essere rieseguite nelle stesse condizioni della versione precedente, in modo da consentire un confronto attendibile tra baseline e algoritmo modificato.

Capitolo 4

Design e implementazione

4.1 Strategia di integrazione nel criterio di clustering

La soluzione proposta introduce una penalità di mobilità basata su due componenti:

- **Componente angolare (COG):** misura la differenza di direzione tra due nodi. Calcola l'angolo più breve tra le due rotte, normalizzato su un intervallo di $[0, 1]$, dove 0 rappresenta nodi che si muovono nella stessa direzione e 1 rappresenta nodi che si muovono in direzioni opposte.
- **Componente di velocità (SOG):** misura il divario della velocità di movimento tra due navi rapportato a una soglia massima tollerata di 25 nodi. Anche questo valore è normalizzato in $[0, 1]$.

Penalità unificata: Le due componenti vengono combinate in una penalità di mobilità unica tramite una media ponderata, dove la componente angolare riceve un peso maggiore (60%) rispetto a quella di velocità (40%).

Metrica finale: La penalità di mobilità viene applicata alla metrica originale di trasmissione attraverso un fattore moltiplicativo. In questo modo, i percorsi

tra nodi con mobilità molto differente vengono penalizzati, aumentando il costo computazionale degli algoritmi di elezione del leader e selezione dei relè.

L'introduzione della metrica consapevole della mobilità influenza i contesti dell'algoritmo di clustering attraverso:

- il calcolo della distanza topologica verso la stazione base, fondamentale per l'elezione del leader;
- la misurazione delle distanze intra-cluster tra leader e nodi membri;
- la selezione dei relè inter-cluster per il routing dei dati.

Questo garantisce che siano preferiti nodi con mobilità simile per mantenere le comunicazioni.

4.2 Estrazione e integrazione delle feature AIS aggiuntive

I parametri principali estratti dai messaggi AIS includono:

- SOG.
- COG.
- ID della nave.
- Posizione geografica (longitudine e latitudine).
- Timestamp del messaggio.

Nel formato AIS raw NMEA 0183 utilizzato in questo lavoro, SOG e COG sono codificati secondo le specifiche standard:

- SOG: Codificato come valore intero in 1/10 di nodo (ovvero moltiplicato per 10). Il valore 1024 viene utilizzato per indicare "dato non disponibile".

- COG: Codificato come valore intero in 1/10 di grado (ovvero moltiplicato per 10). Il valore 3600 (equivalente a 360.0 gradi) viene utilizzato per indicare “dato non disponibile”.

Durante la fase di parsing, questi valori vengono decodificati dividendo per i rispettivi fattori di scala per ottenere i valori in unità standard (nodi e gradi).

4.2.1 Decodifica messaggi AIS

Per la decodifica dei messaggi AIS, è stata utilizzata la libreria `dk.dma.ais`, sviluppata dal Danish Maritime Authority (DMA) [Dan]. Questa libreria fornisce un parser in grado di gestire il protocollo NMEA 0183 standard e la decodifica dei messaggi AIS multi-riga.

La libreria è integrata nel progetto e offre le seguenti funzionalità [Dan]:

- parsing dei singoli frame AIS secondo le specifiche NMEA 0183;
- ricomposizione automatica dei messaggi multi-sentenza;
- validazione del checksum;
- decodifica dei campi contenuti nei messaggi.

Nel nostro caso (listing 4.1), la libreria fornisce accesso ai campi di velocità e rotta attraverso l'interfaccia `IVesselPositionMessage`, che rappresenta i messaggi AIS contenenti informazioni di posizionamento (tipicamente messaggi di tipo 1, 2, 3) [Dan]. Si controlla se il messaggio attuale contenga SOG e COG e, se non li contiene, viene assegnato loro il valore `null`.

4.2. ESTRAZIONE E INTEGRAZIONE DELLE FEATURE AIS AGGIUNTIVE

Listing 4.1: Parte del codice della classe AisPayload.

```
1 data class AisPayload(  
2     val boatId: Int,  
3     val timestamp: Instant,  
4     val longitude: Double,  
5     val latitude: Double,  
6     val speedOverGround: Double?,  
7     val courseOverGround: Double?,  
8 ) {  
9     companion object {  
10         fun from(  
11             boatId: Int,  
12             timestamp: Instant,  
13             aisMessage: AisMessage,  
14         ): AisPayload? =  
15             if (  
16                 aisMessage is IPositionMessage &&  
17                 ParsingUtils.validateLongitude(aisMessage.pos.longitudeDouble) &&  
18                 ParsingUtils.validateLatitude(aisMessage.pos.latitudeDouble)  
19             ) {  
20                 // Se messaggio non contiene sog e cog, questi diventano null  
21                 val vesselPositionMessage = aisMessage as? IVesselPositionMessage  
22                 AisPayload(  
23                     boatId,  
24                     timestamp,  
25                     aisMessage.pos.longitudeDouble,  
26                     aisMessage.pos.latitudeDouble,  
27                     if (vesselPositionMessage?.isSogValid == true)  
28                         vesselPositionMessage.sog / 10.0 else null,  
29                     if (vesselPositionMessage?.isCogValid == true)  
30                         vesselPositionMessage.cog / 10.0 else null,  
31                 )  
32             } else {  
33                 null  
34             }  
35         }  
36     }  
37 }
```

4.2. ESTRAZIONE E INTEGRAZIONE DELLE FEATURE AIS AGGIUNTIVE

Listing 4.2: Estensione aggiunta alle tracce GPX delle navi.

```
1 <extensions>
2   <sog>XXX.X</sog>
3   <cog>XXX.X</cog>
4 </extensions>
```

4.2.2 Generazione tracce GPX

Successivamente alla conversione dei dati AIS, SOG e COG vengono integrati all'interno dei file GPX generati dai messaggi del Sistema di Identificazione Automatica. Le tracce GPX fungono da ponte tra i dati AIS e la simulazione Alchemist, consentendo di riprodurre i movimenti delle imbarcazioni e allegare metadati aggiuntivi ai nodi.

Nella fase di generazione delle tracce, sono state apportate le seguenti modifiche:

1. *Anonimizzazione del nome*: i nomi dei vascelli (MMSI) vengono sostituiti con identificativi anonimizzati nel formato **anonXXXX**, dove XXXX è un numero progressivo.
2. *Estensioni GPX*: Per ogni punto di traccia (**trkpt**), sono stati aggiunti elementi di estensione contenenti SOG e COG nel formato indicato nel listing 4.2.

4.2.3 Caricamento di SOG e COG nella simulazione

Durante la simulazione Alchemist, l'azione **LoadSogCogFromTrace** (listing 4.3) è responsabile della lettura di SOG e COG dai file GPX e della loro disponibilità come molecole nel simulatore.

L'azione funziona nel seguente modo:

1. associa ogni nodo della simulazione al corrispondente file GPX in base all'indice;
2. per ogni passo di simulazione, individua il punto di traccia corrispondente al tempo di simulazione corrente;

4.2. ESTRAZIONE E INTEGRAZIONE DELLE FEATURE AIS AGGIUNTIVE

3. legge i valori di SOG e COG dalla traccia;
4. filtra i valori non disponibili (`NaN`);
5. assegna i valori come concentrazioni delle molecole `sog` e `cog` nel nodo (listing 4.4).

4.2. ESTRAZIONE E INTEGRAZIONE DELLE FEATURE AIS AGGIUNTIVE

Listing 4.3: Codice semplificato dell'azione Alchemist per leggere SOG e COG dalle tracce GPX.

```
1 class LoadSogCogFromTrace<T>(  
2     private val environment: NavigationEnvironment<T>,  
3     node: Node<T>,  
4     private val path: String,  
5     referenceTimeEpochSeconds: Number,  
6 ) : AbstractAction<T>(node) {  
7  
8     // Inizializzazione dei percorsi e filtro dei file .gpx omessi  
9  
10    private val points by lazy {  
11        val nodeIndex = environment.nodes.indexOf(node)  
12        // Associazione del nodo al file traccia corrispondente  
13        readPoints(files[nodeIndex])  
14    }  
15  
16    override fun execute() {  
17        val time = environment.simulation.time.toDouble()  
18        // Ricerca dell'ultimo punto rispetto al tempo di simulazione  
19        val point = points.lastOrNull { it.time <= time } ?: points.firstOrNull()  
20  
21        // Aggiornamento delle molecole nel nodo (Speed e Course Over Ground)  
22        point?.sog?.takeUnless(Double::isNaN)?.let { set("sog", it) }  
23        point?.cog?.takeUnless(Double::isNaN)?.let { set("cog", it) }  
24    }  
25  
26    private fun readPoints(file: File): List<Point> {  
27        // Setup del parser XML DOM omesso  
28        val xmlPoints = document.getElementsByTagNameNS("trkpt")  
29  
30        return (0 until xmlPoints.length)  
31            .map { xmlPoints.item(it) as Element }  
32            .mapNotNull {  
33                Point(  
34                    // Calcolo del tempo relativo dall'inizio della simulazione  
35                    time = /* parsing Instant e calcolo Duration */,  
36                    sog = it.text("sog")?.toDoubleOrNull() ?: Double.NaN,  
37                    cog = it.text("cog")?.toDoubleOrNull() ?: Double.NaN,  
38                )  
39            }  
40            .filter { it.time >= 0.0 }  
41            .sortedBy { it.time }  
42    }  
43  
44    // Metodi di supporto e data class Point omessi  
45 }
```

Listing 4.4: Utilizzo di LoadSogCogFromTrace nel file di configurazione della simulazione.

```

1 _pools:
2   - pool: &move
3     - time-distribution: 0.1
4       type: Event
5       actions:
6         - type: LoadSogCogFromTrace
7           parameters:
8             path: "navigation-routes"
9             referenceTimeEpochSeconds: *start-time

```

Listing 4.5: Costanti utilizzate per il calcolo della penalità di mobilità.

```

1 private const val COG_WEIGHT = 0.6
2 private const val SOG_WEIGHT = 0.4
3 private const val MAX_TOLERATED_SPEED_DIFF_KNOTS = 25.0
4 private const val MOBILITY_PENALTY_MULTIPLIER = 2.0
5 private const val HALF_TURN_DEGREES = 180.0
6 private const val FULL_TURN_DEGREES = 360.0
7
8 /** Reads an optional Double property from the environment. */
9 private fun CollectiveDevice<*>.getIfDefined(name: String): Double =
10    if (isDefined(name)) get<Double>(name) else Double.NaN

```

4.3 Modifiche all'algoritmo

4.3.1 Costanti definite e lettura dei valori

I valori di SOG e COG vengono letti dall'ambiente di simulazione tramite la funzione `getIfDefined`, che restituisce `NaN` se il parametro non è disponibile.

Il listing 4.5 mostra le costanti utilizzate per il calcolo della penalità di mobilità e la funzione `getIfDefined`. Questi valori sono stati scelti per bilanciare l'influenza della direzione e della velocità sulla metrica di selezione.

Significato dei parametri

- **COG_WEIGHT** e **SOG_WEIGHT**: Il peso della direzione e della velocità nel calcolo della penalità combinata. Si è deciso di attribuire più importanza alla direzione (0.6) che alla velocità (0.4).
- **MAX_TOLERATED_SPEED_DIFF_KNOTS**: La differenza massima di velocità tollerata (25 nodi). Differenze superiori a questo valore sono considerate comple-

Listing 4.6: Calcolo della penalità angolare basata su COG.

```
1 private fun angularMobilityPenalty(  
2     sourceCog: Double,  
3     targetCog: Double,  
4 ): Double = when {  
5     sourceCog.isNaN() || targetCog.isNaN() -> 0.0  
6     else -> {  
7         val diff = abs(sourceCog - targetCog) % FULL_TURN_DEGREES  
8         val angularDistance = if (diff > HALF_TURN_DEGREES) FULL_TURN_DEGREES -  
9             diff else diff  
10         angularDistance / HALF_TURN_DEGREES  
11     }  
}
```

tamente penalizzanti.

- **MOBILITY_PENALTY_MULTIPLIER**: Un fattore di amplificazione (2.0) che rende la penalità di mobilità significativa nel calcolo della metrica finale.

4.3.2 Funzioni di calcolo della penalità

Penalità angolare (COG)

Il listing 4.6 implementa il calcolo della penalità angolare, che quantifica quanto la direzione di un nodo differisce da quella di un suo vicino:

L'algoritmo calcola la minima distanza angolare tra due direzioni, considerando che gli angoli sono circolari ($[0^\circ, 360^\circ)$). Ad esempio, una nave a 350° e una a 10° hanno una differenza di soli 20° e non di 340° . Il risultato è normalizzato in $[0, 1]$, dove 0 indica navi perfettamente allineate e 1 indica navi con direzioni opposte.

Penalità di velocità (SOG)

Il listing 4.7 implementa il calcolo della penalità di velocità:

Questa funzione calcola la penalità come il rapporto tra la differenza assoluta di velocità tra i due nodi e una soglia di tolleranza. La penalità è limitata a 1,0 se la differenza eccede la soglia, poiché differenze maggiori non fornirebbero informazioni aggiuntive utili. Il risultato è quindi sempre in $[0, 1]$, coerentemente con la penalità angolare descritta nella sezione precedente (sezione 4.3.2).

Listing 4.7: Calcolo della penalità di velocità basata su SOG.

```
1 private fun speedMobilityPenalty(  
2     sourceSog: Double,  
3     targetSog: Double,  
4 ): Double = when {  
5     sourceSog.isNaN() || targetSog.isNaN() -> 0.0  
6     else -> (abs(sourceSog - targetSog) / MAX_TOLERATED_SPEED_DIFF_KNOTS).coerceIn  
7         (0.0, 1.0)  
8 }
```

4.3.3 Integrazione nella metrica di selezione

Calcolo della penalità combinata

Il listing 4.8 mostra le modifiche effettuate nell'entry point della simulazione. Vengono lette la velocità e la direzione della nave attuale dalla simulazione e si ottiene tramite `neighboring` un campo contenente SOG e COG del proprio vicinato. In seguito, tramite le funzioni mostrate nella sezione 4.3.2 e nella sezione 4.3.2, vengono calcolate le penalità. Queste ultime vengono unite e ponderate (dando più importanza alla direzione) e viene calcolata la metrica finale `mobilityAwareMetric`. La metrica finale è ottenuta moltiplicando il tempo di trasmissione originale per un fattore che incorpora la penalità di mobilità. Quindi, nel caso peggiore, per due navi con direzioni opposte e velocità molto diverse, la metrica arriva a tre volte `timeToTransmit`. Se invece non ci sono differenze significative, oppure se SOG e COG non sono disponibili, la metrica resta equivalente a `timeToTransmit`.

Listing 4.8: Modifiche inserite nell'algoritmo della simulazione.

```
1 val mySog = environment.getIfDefined("sog")
2 val myCog = environment.getIfDefined("cog")
3 val neighborSogs = neighboring(mySog)
4 val neighborCogs = neighboring(myCog)
5
6 // Calcolo delle penalizzazioni per ogni vicino
7 val angularPenalty = neighborCogs.map { neighborCog ->
8     angularMobilityPenalty(myCog, neighborCog.value)
9 }
10 val speedPenalty = neighborSogs.map { neighborSog ->
11     speedMobilityPenalty(mySog, neighborSog.value)
12 }
13
14 // Combinazione pesata delle penalizzazioni
15 val combinedMobilityPenalty = angularPenalty.alignedMapValues(speedPenalty) {
16     cogPen, sogPen ->
17     (cogPen * COG_WEIGHT) + (sogPen * SOG_WEIGHT)
18 }.inject(environment, "mobility-penalty")
19
20 // Applicazione della penalizzazione alla metrica originale
21 val mobilityAwareMetric = timeToTransmit.alignedMapValues(combinedMobilityPenalty)
22     { time, penalty ->
23     time * (1.0 + (penalty * MOBILITY_PENALTY_MULTIPLIER))
24 }.inject(environment, "mobility-aware-metric")
```

Applicazione nei calcoli

La metrica consapevole della mobilità è applicata principalmente in tre punti dell'algoritmo (listing 4.9):

La metrica è usata come funzione di costo nell'operatore `distanceTo`, che calcola il gradiente di distanza da ogni nodo verso il ground station. Inoltre è utilizzata per l'elezione dei leader dei cluster e per la scelta dei relay.

Listing 4.9: Utilizzo della nuova metrica nella selezione del leader e del relay.

```
1 // Calcolo della distanza al ground station
2 val clusteredTimeToStation: Double =
3     distanceTo(
4         groundStation,
5         metric = mobilityAwareMetric,
6     ).inject(environment, "clustered-timeToStation")
7
8 // Elezione del leader di cluster
9 val myLeader: Int =
10     boundedElection(
11         strength = -clusteredTimeToStation,
12         bound = streamingBitRate.timeToTransmitOneMb,
13         metric = mobilityAwareMetric,
14         selectBest = { c1, c2 ->
15             maxOf(
16                 c1,
17                 c2,
18                 compareBy<Candidacy<Int, Double, Double>> { it.strength }.thenBy {
19                     it.candidate },
20             ),
21         ).inject(environment, "myLeader")
22
23 // Selezione del relay per la trasmissione
24 val myRelay =
25     potentialRelays
26         .alignedMapValues(timesToStationAround + mobilityAwareMetric) { canRelay,
27             distance ->
28                 when {
29                     canRelay -> distance
30                     else -> POSITIVE_INFINITY
31                 }
32         }.all
33         .fold(localId to POSITIVE_INFINITY) { current, entry ->
34             if (entry.value < current.distanceToLeader) entry.id to entry.value
35             else current
36         }.relayId()
37         .inject(environment, "myRelay")
```

Capitolo 5

Valutazioni

5.1 Risultati dell'esperimento precedente

5.2 Risultati del nuovo algoritmo

5.3 Confronto e discussione

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Limitazioni dello studio

Elenco delle figure

2.1	Livelli di astrazione della programmazione aggregata. Le capacità software e hardware di alcuni dispositivi sono utilizzate per implementare costrutti di FC a livello aggregato. Questi costrutti sono utilizzati per implementare operazioni di coordinamento a blocchi resilienti, che vengono poi racchiuse e combinate insieme per produrre API. © 2015 IEEE. Ristampata, con permesso, da [BPV15].	9
2.2	Prospettiva dinamica di SCR [CPVN19].	11
2.3	Rappresentazione grafica del meta-modello di Alchemist [Pia21]. . .	16

List of Listings

2.1	Esempio di codice Collective per una semplice simulazione di programmazione aggregata.	13
2.2	Parte della configurazione della simulazione per il programma: listing 2.1.	18
4.1	Parte del codice della classe <code>AisPayload</code>	28
4.2	Estensione aggiunta alle tracce GPX delle navi.	29
4.3	Codice semplificato dell'azione Alchemist per leggere SOG e COG dalle tracce GPX.	31
4.4	Utilizzo di <code>LoadSogCogFromTrace</code> nel file di configurazione della simulazione.	32
4.5	Costanti utilizzate per il calcolo della penalità di mobilità.	32
4.6	Calcolo della penalità angolare basata su COG.	33
4.7	Calcolo della penalità di velocità basata su SOG.	34
4.8	Modifiche inserite nell'algoritmo della simulazione.	35
4.9	Utilizzo della nuova metrica nella selezione del leader e del relay. . .	36

Bibliografia

- [AFBPT25] Ghassan Al-Falouji, Martina Baiardi, Danilo Pianini, and Sven Tomforde. Compass: A roadmap to collaborative perception and autonomy in maritime systems. In *2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, pages 69–74, 2025.
- [BPAFT25] Martina Baiardi, Danilo Pianini, Ghassan Al-Falouji, and Sven Tomforde. Robust communication through collective adaptive relay schemes for maritime vessels. In *2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, pages 21–32, 2025.
- [BPV15] Jacob Beal, Danilo Pianini, and Mirko Viroli. Aggregate programming for the internet of things. *Computer*, 48(9):22–30, 2015.
- [BPW14] Marco Balduzzi, Alessandro Pasta, and Kyle Wilhoit. A security evaluation of ais automated identification system. In *Proceedings of the 30th Annual Computer Security Applications Conference, ACSAC '14*, page 436–445, New York, NY, USA, 2014. Association for Computing Machinery.
- [BV14] Jacob Beal and Mirko Viroli. Building blocks for aggregate programming of self-organising applications. In *2014 IEEE Eighth International Conference on Self-Adaptive and Self-Organizing Systems Workshops*, pages 8–13, 2014.

- [BV16] Jacob Beal and Mirko Viroli. Aggregate programming: From foundations to applications. In *Formal Methods for the Quantitative Evaluation of Collective Adaptive Systems*, pages 233–260. Springer, 2016.
- [Col26] Kollektive Organization. Kollektive: Practical. scalable. kotlin-first aggregate programming. <https://kollektive.github.io/>, 2026. Accessed: 2026.
- [CPL15] Alessandro Chiurco, Leonardo Pagnotta, and Francesco Longo. *Ship bridge simulators for training in maritime domain*. PhD thesis, Scuola di Dottorato Pitagora, 2015.
- [CPVN19] Roberto Casadei, Danilo Pianini, Mirko Viroli, and Antonio Natali. Self-organising coordination regions: A pattern for edge computing. In *Coordination Models and Languages: 21st IFIP WG 6.1 International Conference, COORDINATION 2019, Held as Part of the 14th International Federated Conference on Distributed Computing Techniques, DisCoTec 2019, Kongens Lyngby, Denmark, June 17–21, 2019, Proceedings*, page 182–199, Berlin, Heidelberg, 2019. Springer-Verlag.
- [Dan] Danish Maritime Authority. Ais-core (dk.dma.ais). <https://github.com/dma-ais/AisLib>. Accessed: 2026.
- [Int26] International Telecommunication Union (ITU). Technical characteristics for an automatic identification system using time division multiple access in the maritime mobile service. Recommendation Recommendation ITU-R M.1371-6, ITU Radiocommunication Sector, Geneva, Switzerland, 02 2026.
- [Pia21] Danilo Pianini. Simulation of Large Scale Computational Ecosystems with Alchemist: A Tutorial. In Miguel Matos and Fabíola Greve, editors, *Lecture Notes in Computer Science*, volume LNCS-12718 of *Distributed Applications and Interoperable Systems*, pages 145–161, Valletta, Malta, June 2021. Springer International Publishing. Part 5: Invited Paper.

- [PMV13] D Pianini, S Montagna, and M Viroli. Chemical-oriented simulation of computational systems with alchemist. *Journal of Simulation*, 7(3):202–215, August 2013.
- [Uni15] United States Coast Guard. 2015 code of federal regulations implementing the ais provisions of the safety of life at sea (solas) convention and marine transportation security act of 2002, 2015. Includes IMO Resolution MSC.99(73) and U.S. Code Title 46.
- [VAB⁺18] Mirko Viroli, Giorgio Audrito, Jacob Beal, Ferruccio Damiani, and Danilo Pianini. Engineering resilient collective adaptive systems by self-stabilisation. *ACM Trans. Model. Comput. Simul.*, 28(2), March 2018.

Acknowledgements

Optional. Max 1 page.